

Team Members:

Name: Yang Zhao **NUID:** 002301737

Name: Jinyue Sun **NUID:** 002826157

Git hub Link : <https://github.com/YZhao-prog/DSAIPG>

1. Game Description: Connect Four

Connect Four is a two-player game played on a vertical 6-row by 7-column grid. Players alternately drop a disc into one of the columns. The disc occupies the lowest available position in the column. The first player to connect four of their own discs in a row (horizontally, vertically, or diagonally) wins. If the board fills without either player connecting four, the game ends in a draw.

2. Framework Integration

- Package Name: com.phasmidsoftware.dsaipg.projects.mcts.connectfour
- Core Interfaces Used:
 - Game<G>
 - State<G>
 - Move<G>
- Implemented Classes:
 - ConnectFourGame: Defines the game rules and starting state.
 - ConnectFourState: Stores the board, current player, and valid moves.
 - ConnectFourMove: Represents a player move (column drop).
 - MCTS and MCTSNode: Implements generic Monte Carlo Tree Search.
 - ConnectFourMain: Runs a single full game of Connect Four with MCTS.
 - ConnectFourSimulationStats: Simulates 500 games and reports win stats.

3. Monte Carlo Tree Search Design

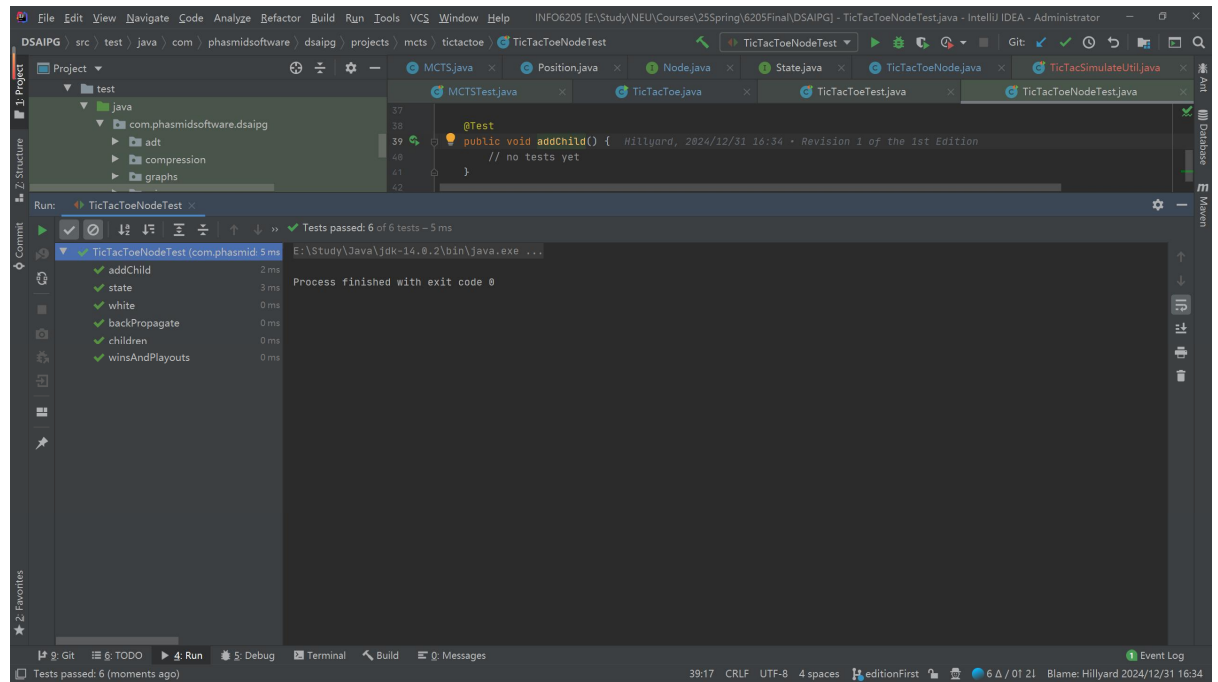
The MCTS algorithm follows the four canonical phases:

- Selection: Traverse child nodes using UCB1 until reaching a leaf node.
- Expansion: Add all valid children (possible moves).
- Simulation: Play a full random game (default rollout policy).
- Backpropagation: Update win/visit statistics on the path to root.
- Number of Simulations per Move: 100
- Exploration Constant: $\sqrt{2}$

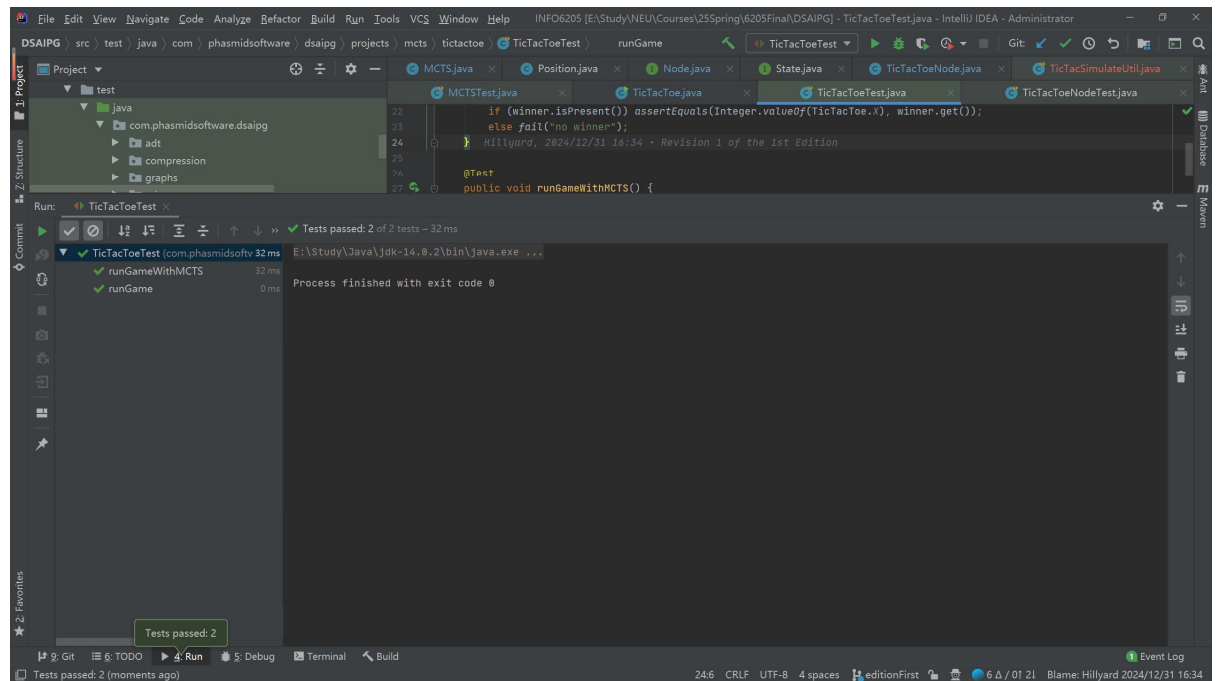
4. Unit Test

- **Unit tests for TicTacToe:**

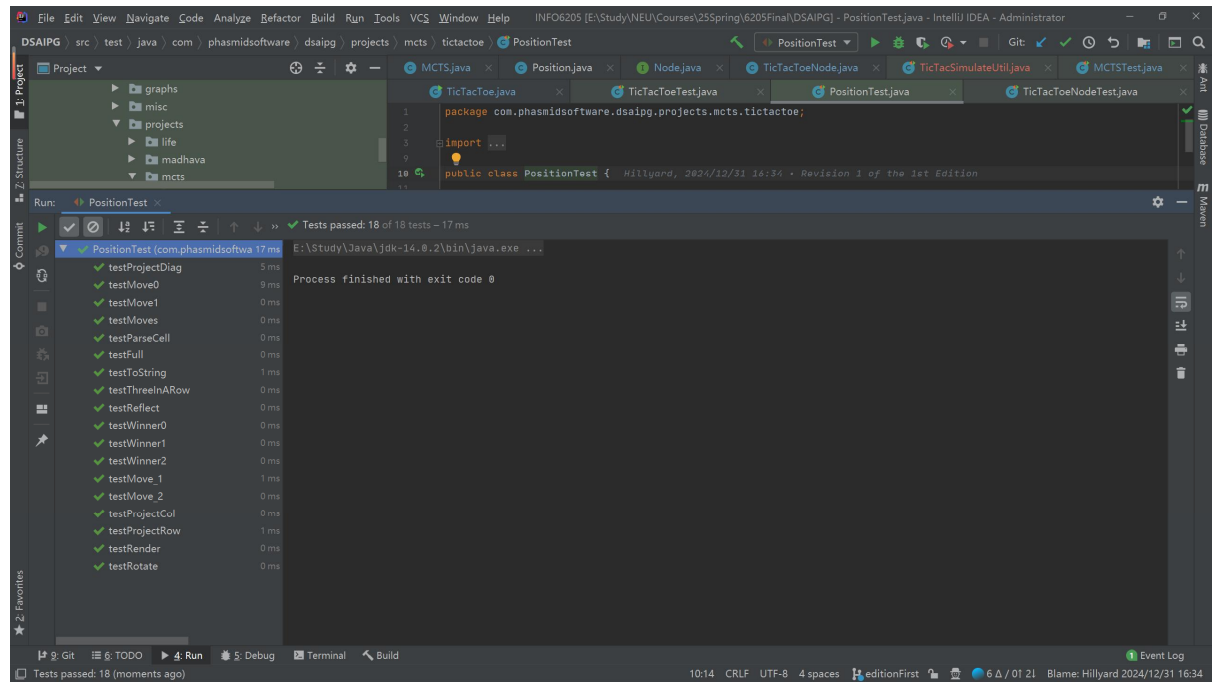
■ TicTacToeNode:



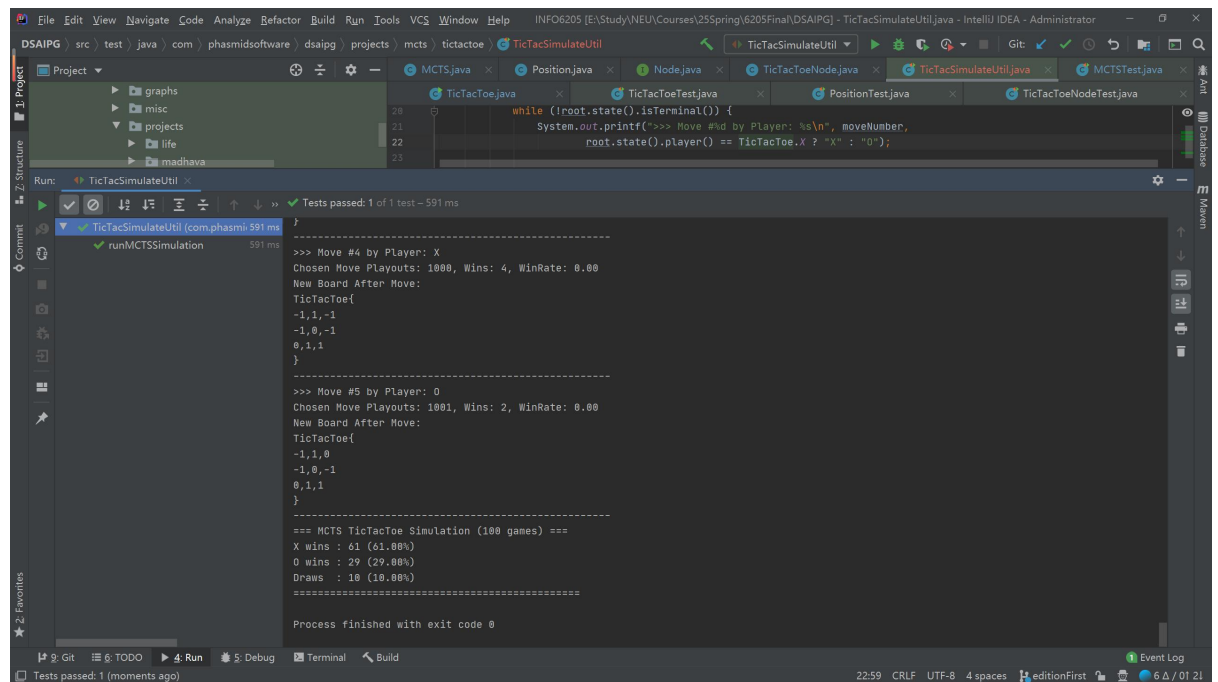
■ TicTacToe:



■ Position

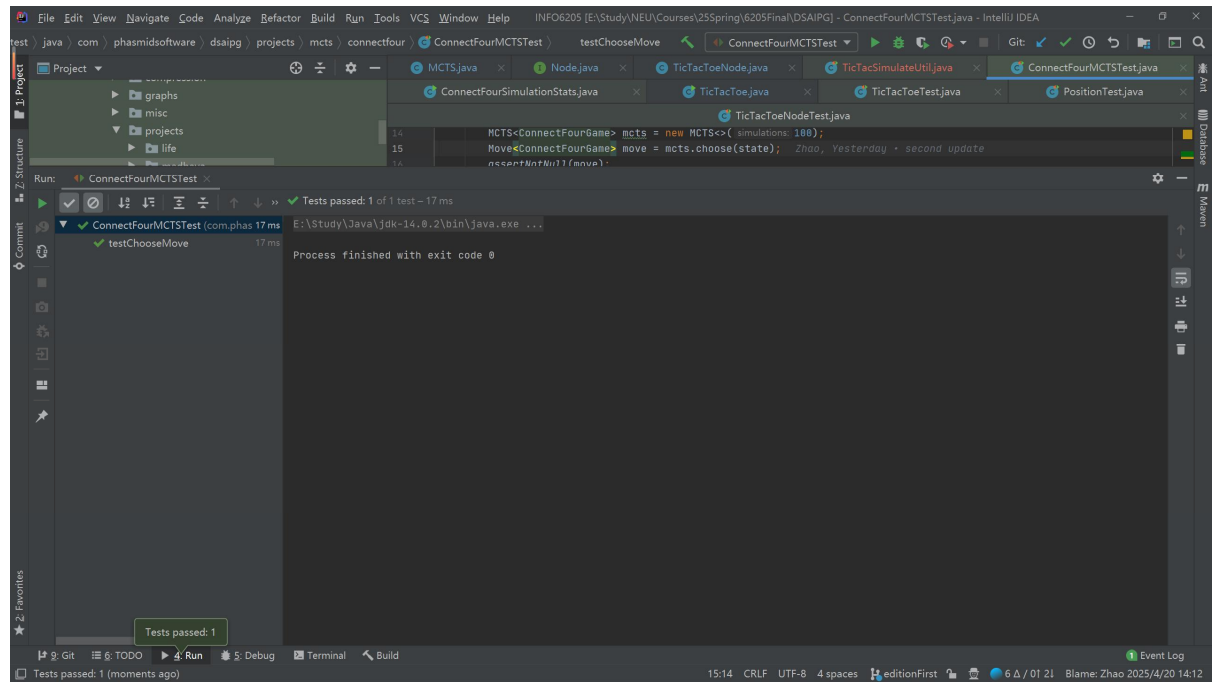


■ TicTacToe - MCTS

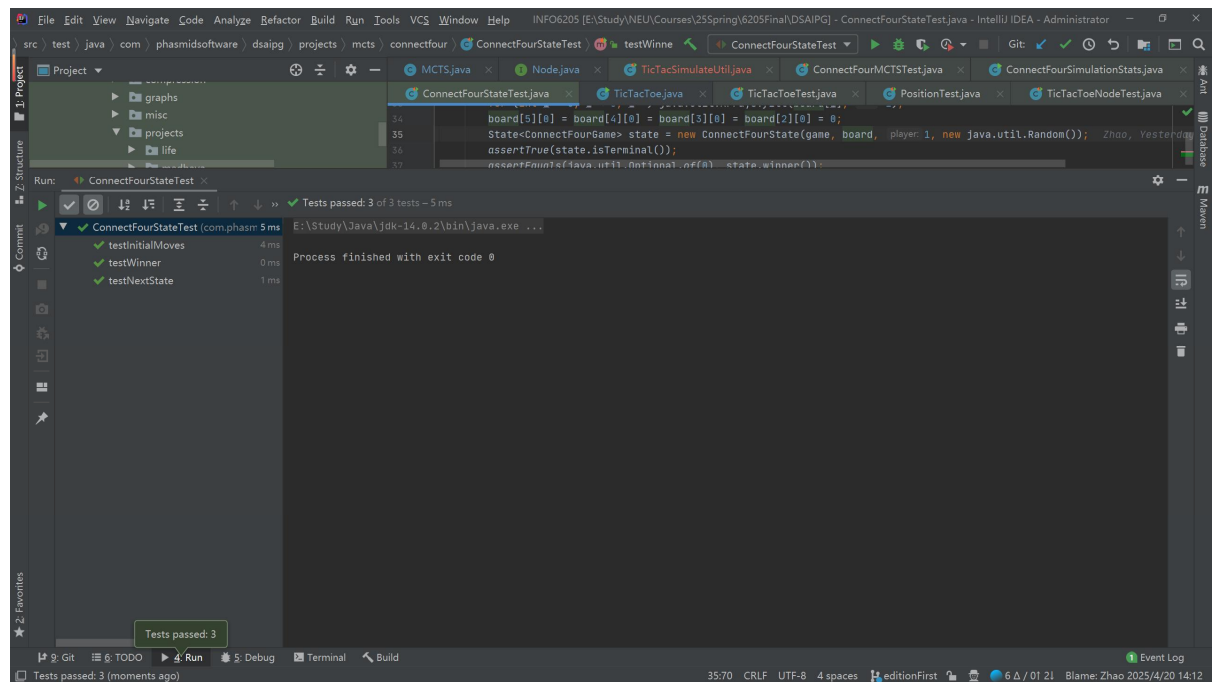


● Unit tests for Connect 4:

■ Connect4Move



■ Connect4State



■ ConnectFourSimulationStats

The screenshot shows the IntelliJ IDEA interface with the `ConnectFourSimulationStats.java` file open. The code defines a `main` method that runs simulations for different strategies. The Run console at the bottom displays the following output:

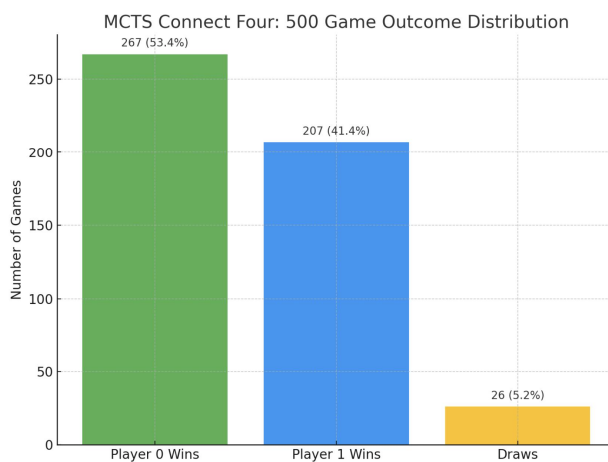
```

Run: ConnectFourSimulationStats
DFAWS: 17 (5.7%)

Strategy: MCTS(100) vs Random
Player 0 Wins: 299 (99.7%)
Player 1 Wins: 1 (0.3%)
Draws: 0 (0.0%)
-----
Strategy: Random vs MCTS(100)
Player 0 Wins: 3 (1.0%)
Player 1 Wins: 296 (98.7%)
Draws: 1 (0.3%)
-----
Strategy: Random vs Random
Player 0 Wins: 163 (54.3%)
Player 1 Wins: 137 (45.7%)
Draws: 0 (0.0%)
-----
Strategy: MCTS(10) vs MCTS(200)
Player 0 Wins: 1 (0.3%)
Player 1 Wins: 299 (99.7%)
Draws: 0 (0.0%)
-----
Process finished with exit code 0
  
```

5. Results

500 Simulated MCTS Games:



- Player 0 Wins: 267 (53.4%)
- Player 1 Wins: 207 (41.4%)
- Draws: 26 (5.2%)

These results demonstrate a slight but expected advantage for Player 0 (first mover). This aligns with the game-theoretic nature of Connect Four.

6. Files and Structure

src/main/java/com/phasmidsoftware/dsaipg/projects/mcts/connectfour/

- ├── ConnectFourGame.java
- ├── ConnectFourMove.java
- ├── ConnectFourState.java

```

├── ConnectFourMain.java
├── MCTS.java
├── MCTSNode.java
└── ConnectFourSimulationStats.java

```

```
src/test/java/com/phasmidsoftware/dsaipg/projects/mcts/connectfour/
```

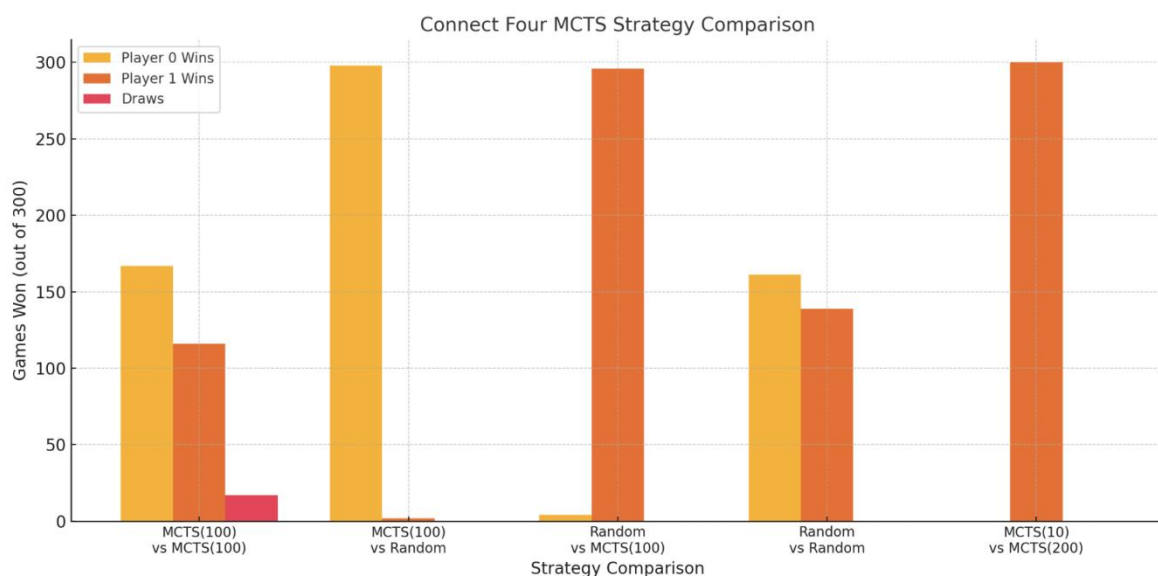
```

├── ConnectFourStateTest.java
├── ConnectFourMCTSTest.java
└── ConnectFourSimulationStats.java

```

7. Performance Analysis

To evaluate the effectiveness of our MCTS implementation under different configurations and against other strategies, we simulated 300 full games under each matchup. Below are the results:



These results demonstrate:

- **MCTS is highly effective** against random strategies.
- **More simulations yield significantly better performance.**
- **A clear first-player advantage** is evident in all settings.
- Even minimal MCTS search (e.g., 100 sims) dramatically outperforms random play.

MCTS's success scales with simulation count and has a decisive edge over random strategies. Strategic tuning (e.g., rollout policy, move ordering) offers potential future gains.

8. Conclusion

This project successfully applied MCTS to Connect Four, demonstrating correct algorithm behavior and strategic balance through simulation. The modular architecture supports extension to other games, and performance can be improved via heuristic rollouts, move ordering, or parallelization.

9. Citations:

[Monte Carlo Tree Search Wiki](#)

<https://www.youtube.com/watch?v=UXW2yZnd17U>